Now we have two vehicle objects that are not particularly related to each other but share the common trait of possessing an engine. The `use Engine;` statement functions like a copy-and-paste when PHP parses the code: the trait code is placed inside the class as if it had been written there in the first place. This actually leads to some idiosyncratic behavior: a class that uses a trait cannot itself also declare any of the properties or methods of the trait:

```
class SaturnFive
{
    use Engine;

    protected int $fuelLevel = 500_000;
}

Output:
SaturnFive and Engine define the same property ($fuelLevel) in the
composition of SaturnFive
```

The key point here is the word "composition." When a trait is used in a class PHP simply adds in the code, as far as it is concerned, the definition of `SaturnFive` is this:

```
class SaturnFive
{
    // use Engine trait code is embedded into SaturnFive class here...
    protected int $fuelLevel = 0;

    // the rest of the Engine trait members, followed by...

    protected int $fuelLevel = 500_000;
}
```

One workaround for this is to have the trait in a parent class, then the child is free to overload in the normal way. For example:

```
class Rocket
{
    use Engine;
}
```